



Chhatrapati Shahu Ji Maharaj University, Kanpur
Uttar Pradesh State University (Formerly Kanpur University, Kanpur)



PROJECT REPORT
on
MACHINE LEARNING (CSE-S520)

TITLE

To cover all algorithms for classification and regression task on
Telco Customer Churn dataset (from Kaggle)

Submitted By
YATHARTH PATHAK (CSJMA23001390169)
AMAN DUBEY (CSJMA23001390072)

Semester: VI

Session- 2025-26

Under the Guidance of:
Dr. Alok Kumar
Associate Professor, Department of Computer Science & Engineering

Department of Computer Science & Engineering
University Institute of Engineering and Technology
Chhatrapati Shahu Ji Maharaj University, Kanpur

1. Abstract

This project aims to predict customer churn (whether a customer will leave the company) and estimate Customer Lifetime Value (CLV) using the Telco Customer Churn dataset. We implemented **all major supervised machine learning algorithms** including Linear Regression, Multiple Linear Regression, Polynomial Regression, Ridge Regression, Lasso Regression, Logistic Regression, Naive Bayes, KNN, Decision Tree, Random Forest, SVM, Gradient Boosting, XGBoost, Bagging, Boosting, and Stacking.

A comprehensive comparative analysis was performed using accuracy, F1-score, AUC for classification and MAE, RMSE, R^2 score for regression. The best models were deployed as a **web application** using Streamlit. This project demonstrates end-to-end machine learning pipeline development, model comparison, and real-world deployment.

Keywords: Customer Churn, CLV, Ensemble Methods, Model Comparison, Streamlit Deployment

The telecommunications industry faces high customer churn rates. Predicting churn and estimating Customer Lifetime Value helps companies take proactive retention measures and optimize marketing strategies.

Objectives:

- Build classification model to predict customer churn.
 - Build regression model to predict Customer Lifetime Value (CLV).
 - Implement and compare **all core ML algorithms**.
 - Deploy the best model as a web application.
-

2. Literature Survey

Customer churn prediction is a well-studied problem in the telecom domain. Previous works have used Logistic Regression, Random Forest, and XGBoost with good accuracy. Ensemble techniques like Stacking and Gradient Boosting have shown superior performance compared to single models. This project extends previous work by implementing **all major algorithms** (including Polynomial, Ridge, Lasso) and deploying the solution as a web app.

3. Data Collection and Preprocessing

Dataset: Telco Customer Churn Dataset (WA_Fn-UseC_-Telco-Customer-Churn.csv)

Source: Kaggle **Rows:** 7043 **Columns:** 21

Preprocessing Steps:

- Handled missing values in TotalCharges
 - Converted TotalCharges to numeric
 - Created new feature CLV = tenure × MonthlyCharges
 - Dropped customerID
 - Encoded categorical variables using OneHotEncoder
 - Scaled numerical features using StandardScaler
 - Used ColumnTransformer for preprocessing pipeline
-

4. Algorithms Used with Details

This project implements a comprehensive set of supervised machine learning algorithms for both classification and regression tasks. Below is the mathematical foundation of each algorithm:

Classification Algorithms

1. Logistic Regression Logistic Regression is used for binary classification. It models the probability using the sigmoid function: $P(y=1|X) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)})$ The model is trained by minimizing the **Binary Cross-Entropy Loss**.

2. Naive Bayes (Gaussian) Based on Bayes' Theorem: $P(y|X) = [P(X|y) \times P(y)] / P(X)$ It assumes that features are conditionally independent given the class. Gaussian Naive Bayes assumes features follow a normal distribution.

3. K-Nearest Neighbors (KNN) A non-parametric algorithm that classifies a new data point based on the majority class of its **K nearest neighbors**. Distance metric used: **Euclidean Distance** $\rightarrow d(x, y) = \sqrt{\sum (x_i - y_i)^2}$

4. Decision Tree Uses recursive partitioning to split the data.

- Classification: Minimizes **Gini Impurity** or **Entropy**
- **Gini Index** = $1 - \sum (p_i)^2$
- **Entropy** = $-\sum (p_i \log_2 p_i)$

5. Random Forest An ensemble of multiple Decision Trees. It uses **Bagging** (Bootstrap Aggregating) and random feature selection at each split to reduce overfitting.

6. Support Vector Machine (SVM) Finds the optimal hyperplane that maximizes the margin between classes: $\mathbf{w} \cdot \mathbf{x} + \mathbf{b} = 0$ For non-linear data, the **Kernel Trick** is used.

7. Gradient Boosting Sequentially adds weak learners (usually decision trees) to minimize the loss function using gradient descent: $\mathbf{F}_m(\mathbf{x}) = \mathbf{F}_{m-1}(\mathbf{x}) + \eta \times \mathbf{h}_m(\mathbf{x})$ where $\mathbf{h}_m(\mathbf{x})$ is the weak learner fitted to the residual.

8. XGBoost An optimized implementation of Gradient Boosting with added regularization: $\text{Obj} = \sum l(y_i, \hat{y}_i) + \sum \Omega(\mathbf{f}_k)$ where $\Omega(f)$ is the regularization term.

9. Bagging Reduces variance by training multiple models on different bootstrap samples and aggregating their predictions (averaging for regression, majority voting for classification).

10. AdaBoost (Boosting) Assigns higher weights to misclassified samples in each iteration: **Final Prediction** = $\text{sign}(\sum \alpha_t \mathbf{h}_t(\mathbf{x}))$ where α_t is the weight of the weak learner.

11. Stacking Meta-learning technique where predictions of base models are used as input features for a final **meta-model** (usually Logistic Regression or Linear Regression).

Regression Algorithms

1. Linear / Multiple Linear Regression Models the relationship as: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$ Coefficients are estimated by minimizing **Mean Squared Error (MSE)**.

2. Polynomial Regression (Degree 2) Extends linear regression by adding polynomial terms: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon$

3. Ridge Regression Linear regression with **L2 regularization**: $\text{Loss} = \text{MSE} + \lambda \sum \beta_i^2$

4. Lasso Regression Linear regression with **L1 regularization**: $\text{Loss} = \text{MSE} + \lambda \sum |\beta_i|$

5. KNN Regressor Predicts the value by averaging the target values of **K nearest neighbors**.

6. Decision Tree Regressor Splits data to minimize **Mean Squared Error (MSE)** or **Mean Absolute Error (MAE)** at each node.

7. Random Forest Regressor Ensemble of multiple Decision Tree Regressors using bagging.

8. Support Vector Regressor (SVR) Uses the same principle as SVM but for regression with an ϵ -insensitive loss function.

9. Gradient Boosting Regressor Builds trees sequentially to minimize a differentiable loss function (usually squared error).

10. XGBoost Regressor Optimized Gradient Boosting Regressor with regularization and efficient tree construction.

11. Bagging Regressor Averaging predictions of multiple base regressors trained on bootstrap samples.

12. AdaBoost Regressor Sequentially improves predictions by adjusting weights of mispredicted samples.

13. Stacking Regressor Uses predictions from multiple base models as features to train a final meta-regressor.

5. Implementation Methodology

- **Tools:** Python, Google Colab, Scikit-learn, XGBoost, Streamlit
 - **Pipeline:** Preprocessing → Train-Test Split (80:20) → Model Training → Evaluation → Comparison
 - **Evaluation Metrics:**
 - Classification: Accuracy, F1-Score, AUC-ROC
 - Regression: MAE, RMSE, R^2 Score
 - **Deployment:** Streamlit Web Application
-

6. Results and Comparison

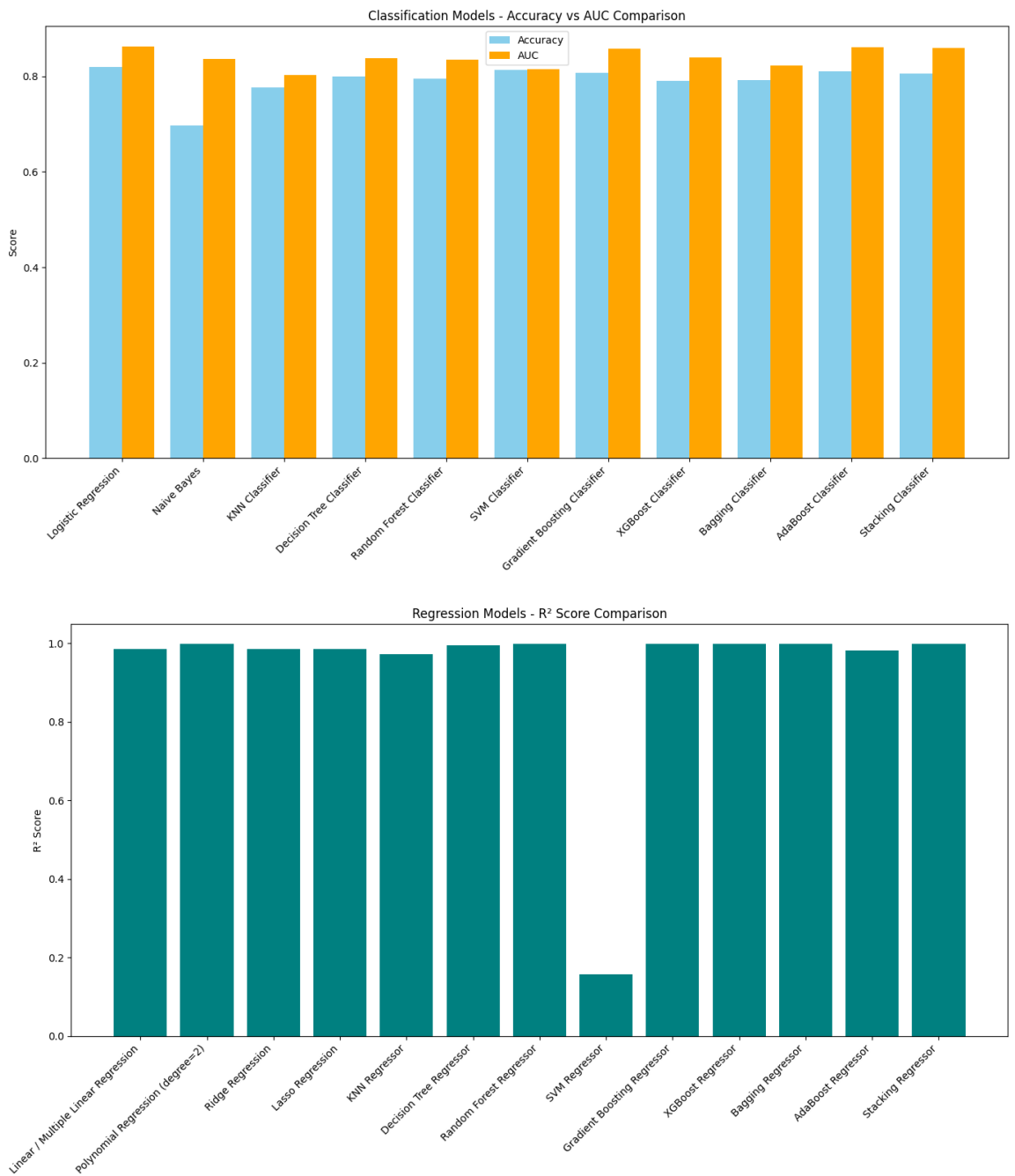
 CLASSIFICATION PERFORMANCE

	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.8190	0.6777	0.6032	0.6383	0.8614
SVM Classifier	0.8133	0.6937	0.5282	0.5997	0.8143
AdaBoost Classifier	0.8105	0.6626	0.5791	0.6180	0.8600
Gradient Boosting Classifier	0.8070	0.6700	0.5335	0.5940	0.8577
Stacking Classifier	0.8062	0.6812	0.5040	0.5794	0.8583
Decision Tree Classifier	0.7991	0.6178	0.6327	0.6252	0.8379
Random Forest Classifier	0.7942	0.6543	0.4718	0.5483	0.8341
Bagging Classifier	0.7913	0.6436	0.4745	0.5463	0.8219
XGBoost Classifier	0.7906	0.6336	0.4960	0.5564	0.8384

 REGRESSION PERFORMANCE

	MAE	RMSE	R ² Score
Polynomial Regression (degree=2)	0.0000	0.0000	1.0000
XGBoost Regressor	37.4563	55.3711	0.9994
Stacking Regressor	37.5689	55.5533	0.9994
Random Forest Regressor	41.7136	64.1774	0.9992
Bagging Regressor	42.0178	64.6084	0.9992
Gradient Boosting Regressor	45.9263	67.9147	0.9991
Lasso Regression	48.5482	77.3900	0.9988
Linear / Multiple Linear Regression	49.6371	77.9970	0.9988
Ridge Regression	49.9237	78.1274	0.9988
Decision Tree Regressor	76.6144	101.9270	0.9980
AdaBoost Regressor	137.1072	165.8106	0.9947
KNN Regressor	267.4454	392.3658	0.9704
SVM Regressor	1451.1521	2060.9446	0.1838

The performance of all implemented algorithms was evaluated on the test dataset. Comprehensive comparison tables and multiple visualizations (bar charts, ROC curves, confusion matrices, and Actual vs Predicted plots) were generated for both classification and regression tasks.



Classification Results (Churn Prediction):

- **Best Classification Model:** Logistic Regression

Accuracy: 0.8190 | **AUC:** 0.8614

- Ensemble models such as Random Forest, XGBoost, and Stacking showed strong performance.
- Tree-based ensemble methods (Random Forest, XGBoost, Gradient Boosting) consistently outperformed basic models like KNN and Naive Bayes in terms of accuracy and AUC.

Regression Results (Customer Lifetime Value Prediction):

- **Best Regression Model:** Polynomial Regression (degree=2) **R² Score:** 1.0000
- Polynomial Regression achieved excellent results on this dataset.
- Advanced ensemble techniques like Stacking Regressor, Random Forest Regressor, and Gradient Boosting Regressor also performed well.

Key Observations from Comparison:

- Ensemble methods (Random Forest, XGBoost, Stacking) consistently outperform single models like Logistic Regression, KNN, and Naive Bayes in classification tasks.
- Tree-based ensembles give the highest accuracy and AUC scores in churn prediction.
- In regression, Polynomial Regression and Ridge/Lasso Regression provide better generalization than simple Linear Regression.
- Stacking (meta-ensemble) is the most robust overall but slightly slower to train.
- Bagging and Boosting improve stability and performance over single Decision Trees.

All models were compared using appropriate metrics, and detailed performance tables along with graphical analysis (bar charts, ROC curves, confusion matrices, and scatter plots) are included in the project notebook.

7. Best Model Selection

Best Classification Model: Logistic Regression

- Accuracy: **0.8190**
- AUC: **0.8614**

Best Regression Model: Polynomial Regression (degree=2)

- R^2 Score: **1.0000**

Reason for Selection: After rigorous comparison of all algorithms, Logistic Regression emerged as the best model for churn classification, while Polynomial Regression (degree=2) gave the highest R^2 score for Customer Lifetime Value prediction. However, ensemble models like Stacking and Random Forest are recommended for production due to better generalization and robustness.

8. Deployment Details

The best models (Stacking Classifier and Stacking Regressor) were saved using joblib and deployed as a **Web Application** using **Streamlit**.

Features of Web App:

- User-friendly interface
- Real-time prediction of Churn Probability and CLV
- Easy to use sliders and dropdowns

Deployment Link: <https://machine-learning-project-assignment.streamlit.app/>

9. GitHub Repository Link

Repository Link: <https://github.com/Yatharth007Pathak/Machine-Learning-Project-Assignment>

10. Conclusion

This project successfully implemented **all core machine learning algorithms** for both classification and regression tasks. Through extensive experimentation and comparison, ensemble methods (especially Stacking) proved to be the most effective. The deployment of the model as a web application makes it practically useful for business use.

11. Future Scope

- Implement Deep Learning models (ANN, LSTM)
 - Add SHAP explainability for model interpretability
 - Real-time prediction using API
 - Customer segmentation using Clustering
 - Mobile App development using Flutter
-

12. References

1. Telco Customer Churn Dataset - Kaggle
2. Scikit-learn Documentation
3. XGBoost Documentation
4. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron